claude-windows / a9fd4426-f234-4e4a-bfb4-1f2c8255c661

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\a9fd4426-f234-4e4a-bfb4-1f2c8255c661\subagents\workflows\wf_076beba8-59c\agent-ab668637b2dbcc293.jsonl`
- SHA-256: `541a92e09190de6610e4b63b76239a5768294ebd030158751b201c0dbd7a4f7c`
- Source modified: `2026-05-31T04:18:43+00:00`
- Imported at: `2026-07-05T16:48:20+00:00`
- project: `wf_076beba8-59c`
- session_id: `a9fd4426-f234-4e4a-bfb4-1f2c8255c661`

Transcript

1. user (2026-05-31T04:17:41.642Z)

Source Extractor

Research question: "Identify the best LOCAL (self-hosted, single NVIDIA CUDA GPU) vision / document-AI / OCR models ? and, as a clearly-flagged approval-gated fallback only, paid cloud API services ? for parsing BANK STATEMENT PDFs into structured transaction tables (date, narration/description, withdrawal/debit, deposit/credit, running balance), for a LOCAL-FIRST personal-finance tool ("muneem").

Target documents: Indian bank statements (HDFC, Axis, ICICI, AU Small Finance) ? dense, multi-column, often SEMI-RULED or borderless tables, sometimes multiple accounts per statement; both born-digital (text layer present) and the harder cases where pdfplumber's extract_tables/coordinate clustering is unreliable (e.g. Axis, whose column layout varies between statements).

HARD CONSTRAINTS (a model that violates these is unusable for us):
1. MUST run 100% LOCALLY for statement contents (highly sensitive). NO cloud APIs by default. Paid/cloud is acceptable ONLY as an explicitly-approved fallback ? still report the best cloud options but label them clearly as "cloud/sensitive ? approval-gated, not default".
2. MUST be usable from Python (transformers / vLLM / Ollama / ONNX Runtime / native lib).
3. License MUST be permissive & commercially safe for BOTH the CODE and the MODEL WEIGHTS: MIT, BSD, Apache-2.0, ISC, HPND. AVOID and explicitly FLAG anything GPL/LGPL/AGPL, CC-BY-NC / non-commercial, research-only, or custom "community"/acceptable-use licenses with restrictions. This is critical ? many document-AI models and their training datasets are non-commercial (e.g. LayoutLMv3 weights are CC-BY-NC; Surya and Marker use GPL-or-commercial dual licensing; several VLM weights ship custom restrictive licenses). Verify the WEIGHTS license, not just the repo license.
4. Hardware: a single consumer/prosumer NVIDIA GPU. Give VRAM tiers (~8-12 GB consumer, ~16-24 GB) and which models/quantizations fit each.

Evaluate and compare these LOCAL approaches ? for each give: code license, MODEL-WEIGHTS license (state explicitly), latest release / maintenance status as of 2025-2026, VRAM need + quantization options, Python integration path, and reliability specifically on DENSE FINANCIAL TABLES / multi-column statements:
- Open vision-language models (VLMs) for document/table understanding: Qwen2-VL and Qwen2.5-VL (note per-size license ? which sizes are Apache-2.0 vs restricted), MiniCPM-V 2.6, InternVL2 / 2.5, Microsoft Phi-3.5-vision, Mistral Pixtral, Meta Llama-3.2-Vision (flag the Llama Community License restrictions), Moondream2, GOT-OCR2.0, olmOCR and other OCR-specialized VLMs.
- OCR-specialist / document-AI pipelines: PaddleOCR + PP-Structure (table recognition), docTR, EasyOCR, Tesseract (already used), Microsoft Table Transformer / TATR (check code AND PubTables-1M dataset/weights license), Donut, Surya (FLAG license), Marker (FLAG license), MinerU / PDF-Extract-Kit, RapidOCR.
- Table-structure recognition specifically (cell/grid reconstruction for borderless tables): TATR, PP-Structure, UniTable, etc.

Also assess: for SEMI-RULED / borderless bank tables, is a prompt-driven VLM (page image -> JSON transactions) more reliable than a classic OCR + table-structure pipeline? What are the accuracy/hallucination risks of VLMs on financial NUMBERS (transposed digits, dropped/duplicated rows), and how should output be VERIFIED? (muneem already machine-checks parses by balance reconciliation ? opening + sum(deposits) - sum(withdrawals) == closing, plus per-row running-balance walks and cross-checks against the bank's printed summary ? so any model's output can be automatically validated, which should inform the recommendation.)

Finally, briefly cover the best PAID/CLOUD fallbacks (clearly flagged approval-gated for sensitive data): AWS Textract (AnalyzeExpense / Tables), Google Document AI, Azure AI Document Intelligence, Reducto, LlamaParse, Anthropic/OpenAI vision ? note any self-hosting option and data-handling terms.

Deliver a RANKED shortlist of the most promising LOCAL options for muneem (NVIDIA GPU, Python, commercial-license-safe), each with license (code + weights), VRAM tier, maturity, strengths/weaknesses for dense bank-statement tables, and a concrete recommendation: which local model(s) to prototype for muneem's statement parsing and how they'd combine with the existing deterministic parsers + balance-reconciliation verification (e.g. VLM as a fallback/cross-check when deterministic parsing fails to reconcile). Cite sources and call out any license uncertainty explicitly."

Fetch and extract key claims from this source:
URL: https://github.com/opendatalab/OmniDocBench
Title: opendatalab/OmniDocBench ? CVPR 2025 document-parsing benchmark (tables, formulas, reading order)
Found via: OCR + table-structure recognition pipelines (practitioner/implementation) search

Task

1. Use WebFetch to retrieve the page content.
2. Assess source quality: primary research/institution? secondary reporting? blog/opinion? forum? unreliable?
3. Extract 2-5 FALSIFIABLE claims that bear on the research question. Each claim must:
   - be a concrete, checkable statement (not vague generalities)
   - include a direct quote from the source as support
   - be rated central/supporting/tangential to the research question
4. Note publish date if available.

If the fetch fails or the page is irrelevant/paywalled, return claims: [] and sourceQuality: "unreliable".

Structured output only.

2. assistant (2026-05-31T04:17:44.023Z)

I'll fetch the OmniDocBench page and extract key claims relevant to the research question.

3. user (2026-05-31T04:18:04.741Z)

OmniDocBench: Key Facts for Document Parsing Evaluation

(1) What is OmniDocBench?

OmniDocBench is a comprehensive benchmark accepted at **CVPR 2025** developed by OpenDataLab. It is designed to evaluate diverse document parsing in real-world scenarios. The paper is available on arXiv (https://arxiv.org/abs/2412.07626).

(2) Document Coverage

The benchmark encompasses:
- **1,651 PDF pages** across **10 document types** (academic papers, financial reports, newspapers, textbooks, handwritten notes, exam papers, magazines, research reports, presentations, books)

- **5 layout types**: single-column, double-column, three-column, mixed, other
- **5 language types**: English, Simplified Chinese, English-Chinese mixed, and others
- **Rich annotations**: 28 block-level elements (paragraphs, tables, formulas, etc.) and 4 span-level elements
- **Table-specific attributes** with 6 annotation types including merged cells, border styles, background colors, and formula inclusion

(3) Benchmarked Models

Specialized VLMs/OCR Models:
- PaddleOCR-VL-1.5, MinerU2.5, GLM-OCR, Youtu-Parsing, FireRed-OCR, Logics-Parsing-v2, MonkeyOCR series, DeepSeek-OCR, Dolphin variants, dots.ocr, OpenDoc, HunyuanOCR

General VLMs:
- Qwen2-VL (72B), Qwen3-VL (235B), InternVL2/3 series, Gemini 2/3 (Flash/Pro), GPT-4o, GPT-5.2, Kimi K2.5

Pipeline Tools:
- Marker, PaddleOCR PP-StructureV3, Mathpix, Docling, Unstructured, OpenParse, Pix2Text

(4) Table Recognition Results (TEDS Scores)

Top performers on **Table TEDS metric**:

Model	Table TEDS ?
MinerU2.5-Pro	**93.42**
RapidTable (pipeline)	**82.5**
Youtu-Parsing	92.02
PaddleOCR-VL-1.5	91.67
StructEqTable (expert VLM)	75.8
GOT-OCR	74.9
Qwen2-VL-72B	71.0

The benchmark notes: *"RapidTable [achieves] full borders [at] 83.0 [TEDS with] 77.1/85.4 [merged cells performance]."*

(5) License

The benchmark code is released under the **Apache-2.0 license**. The dataset statement notes: *"The PDFs are collected from public online channels and community user contributions... for research purposes only and not for commercial use."*

Contact for copyright concerns: OpenDataLab@pjlab.org.cn

(6) Pipeline vs End-to-End VLM Comparison

The leaderboard structure suggests specialized document-parsing VLMs outperform general-purpose models. MinerU2.5-Pro (1.2B specialized VLM) achieves **Overall 95.75**, significantly higher than GPT-5.2 (236.59) or Kimi K2.5 (84.53). However, the benchmark states: *"End-to-end evaluation assesses the model's accuracy in parsing PDF page content."* Both approaches are evaluated on identical metrics (Edit Distance, TEDS, CDM for formulas).

4. assistant (2026-05-31T04:18:08.672Z)

I have a comprehensive extraction. Let me note one anomaly in the fetched data (the "GPT-5.2 (236.59)" figure appears garbled/implausible ? edit distance values, and the model name itself is suspect given the knowledge cutoff). Let me verify the license and dataset terms directly since the license is a hard constraint for muneem, and cross-check key numbers.

5. user (2026-05-31T04:18:14.922Z)

License and Usage Information for OmniDocBench

(1) CODE License

The code is released under the **Apache-2.0 license**, as indicated in the repository footer.
The exact license file reference states: "[Apache-2.0 license](#Apache-2.0-1-ov-file)"

(2) DATASET/PDFs Terms

The dataset is **research-only, non-commercial**. The exact statement reads:

"The dataset is for research purposes only and not for commercial use."

Additionally: "If there are any copyright concerns, please contact OpenDataLab@pjlab.org.cn"

(3) Financial Documents Category

Yes, financial reports are explicitly mentioned as a document category. The exact quote:

"It encompasses a wide range of content, including academic papers, financial reports, newspapers, textbooks, and handwritten notes."

The benchmark includes the category "research_report" which encompasses "Research reports and financial reports" per the attribute labels section.

6. assistant (2026-05-31T04:18:22.807Z)

I now have verified information. The license distinction is clear: code is Apache-2.0 (a benchmark, not a deployable model), but the dataset is research-only/non-commercial. The benchmark explicitly includes financial reports as a document category and reports table-recognition (TEDS) scores ? both directly relevant to muneem.

Note on data quality: the first fetch returned some implausible/garbled figures (e.g., "GPT-5.2 (236.59)", "Kimi K2.5", "Qwen3-VL (235B)") that conflate edit-distance and overall scores and reference models beyond plausible knowledge ? likely the small fetch model misreading a dense, evolving leaderboard. I'll only surface claims that are stable and verifiable (the benchmark's nature, scope, table-metric usage, and license), and explicitly flag the leaderboard-number uncertainty rather than asserting specific shaky numbers.

This is a primary source: the official repository for a CVPR 2025 benchmark from OpenDataLab (Shanghai AI Lab).

7. user (2026-05-31T04:18:33.063Z)

Structured output provided successfully

8. assistant (2026-05-31T04:18:43.274Z)

Extraction complete. Summary of findings for the source at https://github.com/opendatalab/OmniDocBench:

Source quality: primary (official GitHub repo for a CVPR 2025 benchmark from OpenDataLab / Shanghai AI Lab; arXiv:2412.07626, dated 2024-12-10).

Relevance to muneem's research question: high. OmniDocBench is a directly applicable evaluation harness for choosing a local document-parsing model, because it (a) explicitly includes financial reports as a document category, (b) scores table-structure reconstruction with a dedicated Table TEDS metric, (c) annotates borderless/merged-cell table attributes (the exact hard case for Axis-style statements), and (d) benchmarks the candidate pipeline tools (PP-StructureV3, RapidTable, Marker, Docling) and VLMs (Qwen2-VL, InternVL, GOT-OCR) on identical metrics.

License posture for muneem: code is Apache-2.0 (safe to use as an eval harness), but the

dataset/PDFs are research-only / non-commercial ? so muneem can use the benchmark methodology and consult its rankings, but cannot redistribute or commercially use its corpus, and must independently verify the license of any specific model it picks (OmniDocBench ranks third-party models but ships no weights of its own).

Important caveat flagged in my output: the WebFetch model returned some garbled/implausible leaderboard figures (e.g. "GPT-5.2 (236.59)", "Kimi K2.5", "Qwen3-VL (235B)") that conflate edit-distance and overall-score columns and reference models beyond plausibility. I deliberately excluded specific shaky numeric rankings from the falsifiable claims and limited claims to verifiable, stable facts (benchmark nature, scope, metrics, license). If the parent agent wants exact per-model Table TEDS numbers, those should be pulled from the arXiv paper's results table rather than trusted from this fetch.